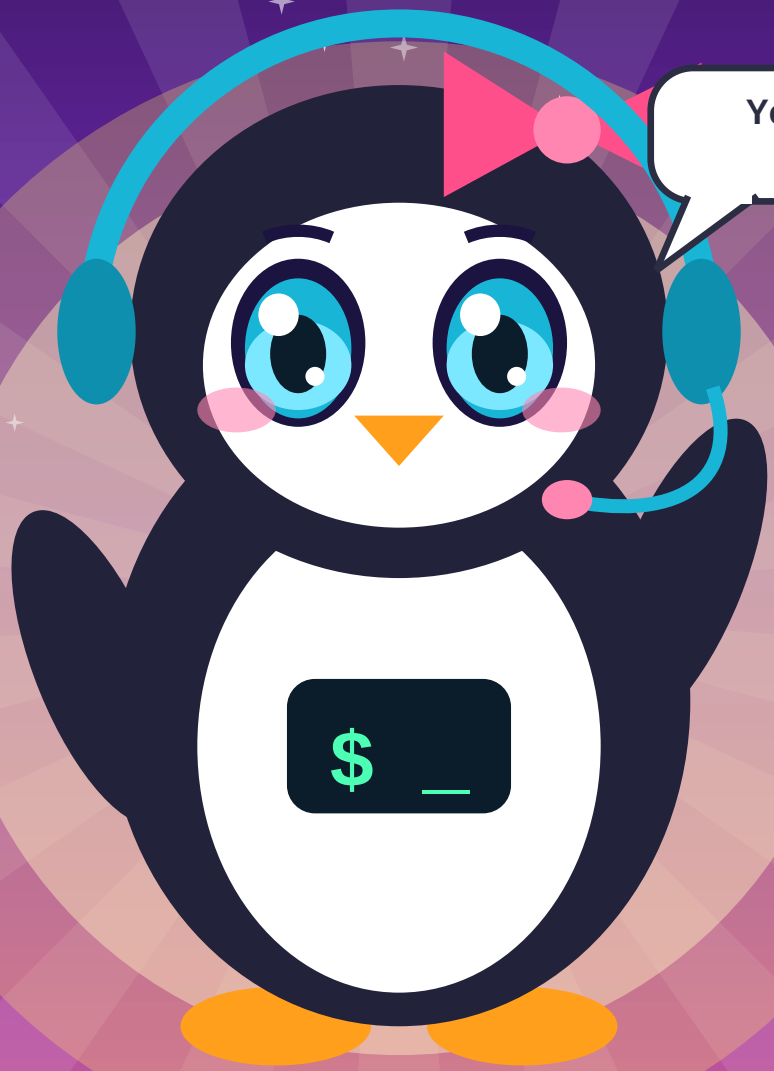
# INTERVIEW BOSS BATTLES V

20 NEW SCENARIOS | INTERNALS | PERFORMANCE | STORAGE | SECURITY




MYTHIC BOSS: THE KERNEL HACKER HP 100%

- CREATED BY -
RUTUJA NAIK

SET 5 * 20 LINUX SCENARIOS * 10 RAPID-FIRE

## 00 HOW TO ANSWER

*TRACE plus measure-then-tune*

| | |
|---|---|
| **T** | **Triage**<br>How bad is it, who is affected, since when, and what changed recently? |
| **R** | **Read the evidence**<br>Check logs, metrics and state before touching anything. |
| **A** | **Analyze by layer**<br>Narrow it down: CPU, memory, disk, network, then application and dependencies. |
| **C** | **Change safely**<br>Smallest fix first, one change at a time, with a rollback ready. |
| **E** | **Educate and prevent**<br>Monitoring, automation, runbook and a blameless post-mortem. |

### YOUR 60-90 SECOND ANSWER TEMPLATE

**1.** Restate the problem and the impact (10 s). **2.** Say your order of hypotheses (10 s). **3.** Walk through 3 to 5 commands and what each tells you (40 s). **4.** State the fix and your rollback plan (15 s). **5.** Finish with prevention: monitoring, automation, documentation (10 s).

### POWER PHRASES THAT SOUND SENIOR

"Let me explain it from the hardware up to the application."
"I'd measure first, then tune one setting at a time."
"I can show that with a command, for example strace or ss."
"I'd compare before and after to prove the fix worked."
"I don't know that one offhand, but here is how I would find out."

### QUEST LOG (CONTENTS)


| | | | | | |
|---|---|---|---|---|---|
| 01 | Explain the Linux boot process | 2 | 11 | Copy 2 TB between servers safely | 12 |
| 02 | What happens when you type ls? | 3 | 12 | Add a new disk and mount it | 13 |
| 03 | Slow app: CPU throttling in containers | 4 | 13 | Filesystem became read-only | 14 |
| 04 | Periodic IO stalls during big writes | 5 | 14 | Connections drop during bursts | 15 |
| 05 | NUMA: uneven performance on 2 sockets | 6 | 15 | Write a production-ready systemd unit | 16 |
| 06 | Batch job starves production | 7 | 16 | Migrate a server to the cloud | 17 |
| 07 | Application crashes with a segfault | 8 | 17 | Thousands of failed SSH logins | 18 |
| 08 | Program hangs and logs nothing | 9 | 18 | Audit for privilege escalation | 19 |
| 09 | Analyse a 10 GB log quickly | 10 | 19 | fork: retry: resource unavailable | 20 |
| 10 | Delete files older than 30 days | 11 | 20 | Choosing ext4, XFS or Btrfs | 21 |

**Troubleshooting** | **Admin and security** | **DevOps and cloud** | then: rapid-fire round and final checklist

# 01 EXPLAIN THE LINUX BOOT PROCESS

ADMIN & SECURITY DIFFICULTY ✦✦✦

## THE QUESTION

**An interviewer asks: walk me through what happens from pressing the power button to the login prompt on a Linux server, and where things can go wrong.**

## SAY IT LIKE THIS (60-90 seconds, out loud)

At power-on, the firmware, BIOS or UEFI, runs a self-test and finds a boot device. With UEFI it loads a bootloader from the EFI System Partition. With BIOS it reads the first sector of the disk. The bootloader, usually GRUB, shows the menu, loads the kernel and the initramfs into memory, and passes kernel parameters. The kernel initialises hardware and mounts the initramfs as a temporary root. That holds the drivers needed to find the real root filesystem, for example for LVM, RAID or encrypted disks. Then it switches to the real root and starts PID 1, which is systemd on modern systems. systemd reads its targets and units, mounts filesystems from fstab, starts networking and services in parallel, and reaches `multi-user.target`, where the login prompt or SSH appears. Failures map to stages: no boot device means firmware or disk, GRUB errors mean the bootloader, a panic or missing root usually means initramfs or drivers, and emergency mode means a failed mount or unit.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | **1. Firmware (BIOS or UEFI): self-test, pick a boot device** | Failure: no bootable device, wrong boot order, dead disk. |
| 2 | **2. Bootloader (GRUB): menu, load kernel and initramfs** | Failure: 'grub rescue', missing config. Fix with grub-install and update-grub from rescue media. |
| 3 | **3. Kernel: initialise hardware, mount the initramfs** | Failure: kernel panic, 'unable to mount root fs'. Check drivers and the root= parameter. |
| 4 | **4. initramfs: load drivers, unlock LUKS, assemble LVM or RAID, mount root** | Failure: dropped to an (initramfs) shell. Rebuild with update-initramfs -u or dracut -f. |
| 5 | **5. PID 1 (systemd): mount fstab, start units, reach the target** | Failure: emergency mode (bad fstab mount) or a failed required unit. |
| 6 | `systemctl get-default and systemd-analyze blame` | The default target, and the slowest units during boot. |
| 7 | `journalctl -b and journalctl -b -1 and dmesg` | Logs for this boot and the previous one, plus kernel messages. |
| 8 | `cat /proc/cmdline and lsblk -f and findmnt` | Kernel parameters, disks and what is mounted where. |

## KEY POINTS TO MENTION

Know the order: firmware, bootloader, kernel, initramfs, systemd, services. Each stage has typical failures and fixes. UEFI uses an EFI System Partition, while BIOS uses the MBR. The initramfs exists so the kernel can find the real root filesystem.

## FIX AND PREVENTION

Keep rescue and recovery steps documented, keep an older kernel installed, test changes to fstab and GRUB config, and use serial console access for cloud and remote servers.

## FOLLOW-UP QUESTIONS THEY MAY ASK

**Q. BIOS versus UEFI?**

A. UEFI is the modern firmware: it boots from an EFI System Partition, supports GPT disks over 2 TB, Secure Boot and faster startup. BIOS uses the MBR and the disk's first sector.

**Q. What does systemd do as PID 1?**

A. It starts and supervises all other processes, mounts filesystems, starts services in parallel by dependency, and reaps orphaned processes.

## AVOID THIS MISTAKE

Memorising the stages without knowing a real failure and fix for each.

# 02 WHAT HAPPENS WHEN YOU TYPE LS?

ADMIN & SECURITY DIFFICULTY ✦✦✦

## THE QUESTION

**What exactly happens when you type ls in a terminal and press Enter?**

## SAY IT LIKE THIS (60-90 seconds, out loud)

The terminal sends the keystrokes to the shell, which parses the line. First it checks whether `ls` is an alias, a function or a shell builtin. If not, it searches each directory in `$PATH` for an executable called `ls`. Once found, the shell calls `fork()` to create a child process, a copy of itself. The child then calls `execve()` to replace its memory with the `ls` program, and the dynamic linker loads shared libraries such as libc. `ls` makes system calls: `openat` and `getdents64` to read the directory, and `write` to print to standard output, which is connected to the terminal. When it finishes, it exits with a status code. The parent shell collects it with `wait()`, stores it in `$?`, and shows the prompt again. I can prove every step with `type` and `strace`.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `type -a ls` | Is it an alias, builtin, function or file? Shows every match in lookup order. |
| 2 | `echo $PATH and which ls` | The directories searched left to right, and the first match. |
| 3 | `strace -f -e trace=execve,openat,getdents64,write ls` | Shows fork and exec, files opened and the directory read, as system calls. |
| 4 | `ldd $(which ls)` | Shared libraries the dynamic linker loads (libc, libselinux and so on). |
| 5 | `ls -l /proc/self/fd` | File descriptors of a process: 0 stdin, 1 stdout, 2 stderr. |
| 6 | `echo $?` | The exit status that the shell collected from the child with wait(). |
| 7 | `ps -o pid,ppid,cmd --forest` | The process tree: the shell is the parent of the command it runs. |
| 8 | **Concepts: fork, exec, wait, file descriptors, PATH** | The building blocks of how every Unix command runs. |

## KEY POINTS TO MENTION

fork copies the process, exec replaces its program, and wait collects the exit status. Standard input, output and error are file descriptors 0, 1 and 2, and the shell connects them to the terminal, files or pipes. Builtins like cd run inside the shell itself, without a fork.

## FIX AND PREVENTION

Practise explaining this with real strace output. It shows you understand system calls instead of memorising words. Mention that pipes and redirection are just the shell rearranging file descriptors before exec.

## FOLLOW-UP QUESTIONS THEY MAY ASK

**Q. Why is cd a shell builtin?**

A. A child process can't change its parent's working directory, so cd must run inside the shell itself.

**Q. What is the difference between fork and exec?**

A. fork creates a new process that is a copy of the caller. exec replaces the current process's program with a new one, keeping the same PID.

## AVOID THIS MISTAKE

Skipping PATH lookup, fork and exec. Those are the heart of the answer.

# 03 SLOW APP: CPU THROTTLING IN CONTAINERS

DEVOPS & CLOUD DIFFICULTY ✦✦✦

## THE QUESTION

**An application in a Kubernetes pod is slow and has high latency, yet node CPU usage is low and the pod's CPU usage looks well below its limit. What is going on?**

## SAY IT LIKE THIS (60-90 seconds, out loud)

That pattern points to CPU throttling by the cgroup quota. Kubernetes CPU limits are enforced by the CFS bandwidth controller: the container gets a quota of CPU time per 100 ms period. A multi-threaded app can burn its whole quota in the first few milliseconds of a period, and then every thread is paused until the next period. So the average CPU looks low, but latency spikes. I'd check the throttling metrics: `container_cpu_cfs_throttled_periods_total` in Prometheus, or `cpu.stat` in the container's cgroup, which shows `nr_throttled` and the throttled time. The fixes are raising or removing the CPU limit while keeping a request, reducing the app's thread counts to match the quota, or setting the runtime's parallelism, for example GOMAXPROCS or JVM options, to the container's CPU limit.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `kubectl describe pod p \| grep -A6 -i limits` | What are the CPU requests and limits? |
| 2 | `kubectl top pod p --containers` | Actual CPU use compared with the limit. |
| 3 | `cat /sys/fs/cgroup/cpu.stat (inside the container, cgroup v2)` | `nr_throttled` and `throttled_usec` show how often and how long it was paused. |
| 4 | `cat /sys/fs/cgroup/cpu.max` | The quota and period, for example `50000 100000` means 0.5 CPU. |
| 5 | `rate(container_cpu_cfs_throttled_periods_total[5m]) / rate(container_cpu_cfs_periods_total[5m])` | PromQL: the fraction of periods that were throttled. High means trouble. |
| 6 | Check thread and worker counts | A runtime that sees all node CPUs may create too many threads for a small quota. |
| 7 | `kubectl set resources deploy app --requests=cpu=500m --limits=cpu=2` | Raise the limit (or remove it) while keeping a request for scheduling. |
| 8 | Set GOMAXPROCS or JVM ActiveProcessorCount to match the limit | Make the runtime's parallelism match the container's quota. |

## COMMON ROOT CAUSES

A low CPU limit with a multi-threaded app, bursty workloads, runtimes that detect the node's CPU count instead of the quota, and many short requests competing for the quota in the same 100 ms period.

## FIX AND PREVENTION

Raise or remove CPU limits for latency-sensitive services (keep requests), tune thread pools, set runtime CPU hints, and monitor the throttling ratio. Use limits mainly to protect shared nodes, not to shape performance.

## FOLLOW-UP QUESTIONS THEY MAY ASK

**Q. Request versus limit for CPU?**

A. The request is what the scheduler reserves and what you get under contention. The limit is a hard cap enforced by throttling.

**Q. What is CFS?**

A. The Completely Fair Scheduler, Linux's default CPU scheduler. Its bandwidth controller implements CPU quotas for cgroups.

## AVOID THIS MISTAKE

Judging CPU limits by average usage graphs alone. Check the throttling counters.

# 04 PERIODIC IO STALLS DURING BIG WRITES

TROUBLESHOOTING DIFFICULTY ✦✦✦

**THE QUESTION**

**Every few minutes the server freezes briefly during heavy file writes and applications stall, though overall disk throughput seems fine. What could cause it?**

**SAY IT LIKE THIS (60-90 seconds, out loud)**

That behaviour fits the Linux writeback mechanism. Writes go into the page cache as dirty pages, and the kernel flushes them to disk in the background. If the application writes faster than the disk can absorb, dirty pages pile up until they reach the thresholds `vm.dirty_background_ratio` and `vm.dirty_ratio`. Past the hard threshold, the kernel throttles or blocks writing processes until data is flushed, which looks like a stall. On servers with a lot of RAM, the default percentages allow gigabytes of dirty data, and a long flush creates a big pause. I'd confirm with `Dirty` and `Writeback` in `/proc/meminfo`, watch `iostat` and `vmstat`, and then tune: lower the thresholds or use the absolute `dirty_bytes` settings, so flushing starts earlier in smaller batches. I'd also check the storage itself for latency, queue depth or cloud volume limits, because that is often the real bottleneck.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `grep -E 'Dirty\|Writeback' /proc/meminfo` | How much dirty data is waiting to be written, and is it spiking? |
| 2 | `sysctl vm.dirty_background_ratio vm.dirty_ratio vm.dirty_expire_centisecs` | Thresholds: when background flushing starts, when writers are blocked, and how old data may get. |
| 3 | `vmstat 1` | `b` (blocked processes), `bo` (blocks out) and `wa` (IO wait) during a stall. |
| 4 | `iostat -xz 1` | Device latency (`await`), queue size and utilization at the time of the stall. |
| 5 | `sudo sysctl -w vm.dirty_background_bytes=268435456 vm.dirty_bytes=1073741824` | Example: start flushing at 256 MB and block writers at 1 GB (absolute values, not a percentage of RAM). |
| 6 | `cat /proc/pressure/io` | PSI: how much time tasks spend stalled on IO. |
| 7 | **Check the storage: cloud volume IOPS or throughput limits, burst credits, RAID write cache** | The disk may simply be slower than the write rate. |
| 8 | **Application side: batch fsync calls, use rate limiting** | Spread writes out instead of producing huge bursts. |

**COMMON ROOT CAUSES**

Large dirty-page thresholds on big-memory servers, bursty writers, slow or throttled storage, fsync storms, a large flush at checkpoint time (databases), and cloud volumes hitting IOPS or throughput limits.

**FIX AND PREVENTION**

Tune the dirty thresholds (absolute bytes), choose faster or provisioned storage, smooth application write patterns, and monitor IO latency and PSI. Test changes under realistic load.

**FOLLOW-UP QUESTIONS THEY MAY ASK**

**Q. Why do big-memory servers suffer more?**

A. Percentage-based thresholds scale with RAM, so a server with 256 GB can build up many gigabytes of dirty data before flushing starts.

**Q. What does fsync do?**

A. It forces a file's dirty data to stable storage before returning, so it survives a crash. Many fsyncs can overload storage.

**AVOID THIS MISTAKE**

Tuning blindly. Measure latency and dirty page counts before and after.

# 05 NUMA: UNEVEN PERFORMANCE ON 2 SOCKETS

TROUBLESHOOTING DIFFICULTY ✦✦✦

## THE QUESTION

**A database on a two-socket server shows inconsistent performance, and some queries are much slower even though CPU and memory look healthy. What do you investigate?**

## SAY IT LIKE THIS (60-90 seconds, out loud)

On multi-socket servers, memory is attached to specific CPUs. That is NUMA, non-uniform memory access. Accessing memory on the local node is fast, but remote memory across the interconnect is slower. If a process runs on one socket but its memory is on the other, or one node runs out of memory while the other has plenty, performance becomes uneven and can even cause swapping or OOM on one node. I'd check the topology with `numactl --hardware` and `lscpu`, then per-node memory with `numastat`. Then I look at how the application is placed. Options are to bind the process and its memory to one node with `numactl --cpunodebind --membind`, interleave memory for large databases with `--interleave=all`, follow the database vendor's NUMA settings, or disable automatic NUMA balancing if it causes overhead. Containers and VMs need CPU pinning set up consistently.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `lscpu \| grep -i numa and numactl --hardware` | Number of NUMA nodes, CPUs per node, memory per node and distances. |
| 2 | `numastat and numastat -p PID` | Per-node allocation and `numa_miss` or `other_node` counters, system-wide and per process. |
| 3 | `head /proc/PID/numa_maps` | Where a process's memory pages actually live. |
| 4 | `numactl --cpunodebind=0 --membind=0 command` | Run a process with CPUs and memory on the same node. |
| 5 | `numactl --interleave=all command` | Spread memory evenly across nodes. Often best for large databases. |
| 6 | `cat /proc/sys/kernel/numa_balancing` | Automatic NUMA balancing: 1 means the kernel migrates pages and tasks. May add overhead. |
| 7 | `taskset -pc PID and systemctl show app -p CPUAffinity` | Current CPU affinity of a process or service. |
| 8 | **Cloud: choose instance types with a single NUMA node, or pin vCPUs consistently** | Avoid surprises on very large instances. |

## COMMON ROOT CAUSES

A process's memory allocated on a remote node, one node's memory exhausted while another is free, threads migrating between sockets, unbalanced interrupt handling, unpinned VMs or containers, NUMA-unaware application settings.

## FIX AND PREVENTION

Bind or interleave according to the workload, size memory per node, follow the database vendor's NUMA guidance, and monitor the `numa_miss` counters. Benchmark, because the right choice depends on the workload.

## FOLLOW-UP QUESTIONS THEY MAY ASK

**Q. What does NUMA mean and why does it exist?**

A. Non-Uniform Memory Access. Each CPU socket has its own memory controller, which scales better than one shared bus, but remote access is slower.

**Q. When do you interleave memory?**

A. For large shared-memory workloads such as databases. It spreads access evenly and avoids exhausting one node.

## AVOID THIS MISTAKE

Pinning everything to one node without checking that the node has enough memory.

# 06 BATCH JOB STARVES PRODUCTION

ADMIN & SECURITY DIFFICULTY ✦✦✦

## THE QUESTION

**A nightly batch job on a shared server uses so much CPU and disk that the production application slows down. How do you protect production without stopping the batch job?**

## SAY IT LIKE THIS (60-90 seconds, out loud)

I'd limit the batch job's resources so production always gets what it needs. The simplest tools are **`nice`** for CPU priority and **`ionice`** for IO priority, to run the job at the lowest priority. For a firm limit, I'd use cgroups through systemd: run the job with **`systemd-run`**, or a unit with **`CPUQuota`**, **`MemoryMax`** and **`IOWeight`**, so it can't use more than its share however busy the server is. I'd also schedule it off-peak, and in a unit set **`Nice=19`** and **`IOSchedulingClass=idle`**. If the problem keeps recurring, the real answer is separation: move the batch to its own server or a replica, so it can't compete with production at all. I'd monitor with PSI and per-cgroup metrics.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `nice -n 19 ionice -c3 ./batch.sh` | Lowest CPU priority and the idle IO class. It only uses spare capacity. |
| 2 | `systemd-run --scope -p CPUQuota=50% -p MemoryMax=2G -p IOWeight=50 ./batch.sh` | Run under cgroup limits through systemd, with a hard cap. |
| 3 | `systemctl show batch.service -p CPUQuota -p MemoryMax` | Verify the limits that were applied to a service. |
| 4 | [Service] Nice=19 CPUQuota=50% MemoryMax=2G IOSchedulingClass=idle | The same limits written into a unit file. |
| 5 | `renice -n 19 -p PID and ionice -c3 -p PID` | Lower the priority of an already running job. |
| 6 | `systemd-cgtop` | Live CPU, memory and IO per control group. |
| 7 | `cat /proc/pressure/cpu and cat /proc/pressure/io` | PSI shows whether production tasks are being stalled. |
| 8 | Schedule with a timer off-peak, or move the job to a separate host | Isolation is the strongest protection. |

## KEY POINTS TO MENTION

nice and ionice are priorities (soft), cgroups are limits (hard). Priorities only help under contention. systemd makes cgroup limits easy. Isolation, using a separate server or replica, is the most reliable fix.

## FIX AND PREVENTION

Apply limits in the unit file, schedule heavy jobs off-peak, monitor pressure metrics, and offload heavy work (read replicas, separate batch nodes) as load grows.

## FOLLOW-UP QUESTIONS THEY MAY ASK

**Q. nice versus cgroup limits?**

A. nice only changes scheduling priority, so a low-priority job can still use idle CPU or hurt IO. cgroup limits cap the usage.

**Q. What is ionice's idle class?**

A. The process gets disk time only when no other process needs it. It can starve if the disk is always busy.

## AVOID THIS MISTAKE

Killing or delaying the job every time. Limit it so both workloads can coexist.

# 07 APPLICATION CRASHES WITH A SEGFAULT

TROUBLESHOOTING DIFFICULTY ✦✦✦

**THE QUESTION**

**An application crashes with 'Segmentation fault' in production. How do you investigate it?**

**SAY IT LIKE THIS (60-90 seconds, out loud)**

A segfault means the process accessed memory it shouldn't, so the kernel sent it SIGSEGV. First I gather evidence: `dmesg` or the journal shows the segfault line with the faulting address and library, which hints at where. Then I get a core dump, a snapshot of the process memory at the crash. With systemd, `coredumpctl` lists and opens crashes, and I analyse the dump with `gdb` and the `bt` backtrace command, ideally with debug symbols installed. I check what changed recently: a new version, a library update, a different environment, bad input or memory exhaustion. Typical causes are null pointer dereferences, buffer overflows, use after free, stack overflow from deep recursion, or incompatible shared libraries. For mitigation I roll back. For the fix I use `valgrind` or AddressSanitizer in a test environment to find the bug.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `dmesg -T \| grep -i segfault` | The segfault line shows the program, faulting address, instruction pointer and library involved. |
| 2 | `coredumpctl list and coredumpctl info PID` | systemd-coredump: recent crashes with signal, time and executable. |
| 3 | `coredumpctl gdb PID then bt full` | Open the core in gdb and print a full backtrace. |
| 4 | `ulimit -c unlimited and cat /proc/sys/kernel/core_pattern` | Are core dumps enabled, and where do they go? |
| 5 | `gdb -batch -ex 'bt' ./app core` | Get a backtrace non-interactively. Needs debug symbols for readable names. |
| 6 | `ldd ./app and apt list --installed \| grep libname` | Library versions: did an update change a dependency? |
| 7 | `valgrind --leak-check=full ./app` | (Test environment) Finds invalid reads and writes, use after free and leaks. |
| 8 | **Roll back and compare: git bisect, previous package version** | Narrow down which change introduced the crash. |

**COMMON ROOT CAUSES**

Null or dangling pointers, buffer overflow, use after free, stack overflow, library version mismatch (ABI), corrupted data or hardware memory errors, and running out of memory in native code.

**FIX AND PREVENTION**

Roll back or patch, capture core dumps and symbols for every release, add AddressSanitizer or fuzz tests in CI, and monitor crash rates. Keep debug symbols available for production binaries.

**FOLLOW-UP QUESTIONS THEY MAY ASK**

**Q. What is SIGSEGV?**

A. The signal the kernel sends when a process accesses invalid memory, such as an unmapped address. The default action is to terminate and dump core.

**Q. What is a core dump?**

A. A file containing a process's memory and registers at the moment it crashed, used for post-mortem debugging.

**AVOID THIS MISTAKE**

Restarting the service in a loop without capturing a core dump or logs.

# 08 PROGRAM HANGS AND LOGS NOTHING

TROUBLESHOOTING DIFFICULTY ✦✦✦

**THE QUESTION**

**A program hangs, uses no CPU, and writes nothing to its logs. How do you find out what it is waiting for?**

**SAY IT LIKE THIS (60-90 seconds, out loud)**

A hung process with no CPU is blocked on something: a lock, a network call, a disk read or a child process. I'd look at its state first with `ps`: `S` means sleeping, `D` is uninterruptible IO wait. Then I attach `strace -p PID` to see the system call it is stuck in, like `futex` (waiting for a lock), `read` or `recvfrom` on a socket, or `connect`. With `lsof -p` or `/proc/PID/fd` I map the file descriptor in that call to a file or network connection, so I see which remote host or file it's waiting on. `/proc/PID/stack` shows the kernel stack, and `gdb` or language tools such as `jstack` or `py-spy dump` show the thread stacks. Then I check the dependency, such as a database, DNS or a mounted share, and fix it or add timeouts.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `ps -o pid,stat,wchan:25,cmd -p PID` | State, and what the kernel is waiting on (the wchan column). |
| 2 | `sudo strace -f -p PID` | The blocked system call: futex, read, recvfrom, connect, poll and so on. |
| 3 | `ls -l /proc/PID/fd and lsof -p PID` | Map the file descriptor from strace to a file or a socket (with remote address). |
| 4 | `ss -tnp \| grep PID` | Network connections of the process and their state. |
| 5 | `sudo cat /proc/PID/stack` | Kernel stack. Useful for D-state processes. |
| 6 | `sudo gdb -p PID -batch -ex 'thread apply all bt'` | Backtraces of all threads (native programs). |
| 7 | `jstack PID or py-spy dump --pid PID` | Thread dumps for Java and Python processes. |
| 8 | **Check the dependency: database, DNS, NFS, an external API** | The hang usually lives in what the program is waiting for. |

**COMMON ROOT CAUSES**

Deadlock or lock contention, waiting on an unresponsive network service, a DNS timeout, hung NFS or disk, missing client timeouts, thread pool exhaustion, or waiting for a child process.

**FIX AND PREVENTION**

Fix or restore the dependency, add timeouts and retries with backoff, add health checks and thread-dump collection, and add metrics for waiting time. Restart only after capturing the evidence.

**FOLLOW-UP QUESTIONS THEY MAY ASK**

**Q. What does futex in strace mean?**

A. A fast user-space mutex: the thread is waiting on a lock or condition variable, so look for deadlock or contention.

**Q. Why use strace -f?**

A. It follows child processes and threads, which matters for multi-threaded programs.

**AVOID THIS MISTAKE**

Killing the process before capturing a thread dump or strace output.

# 09 ANALYSE A 10 GB LOG QUICKLY

DEVOPS & CLOUD DIFFICULTY ✦✦✦

## THE QUESTION

**You need to find the top 10 client IPs and the most common error from a 10 GB web server log, quickly, on a production server. How do you do it?**

## SAY IT LIKE THIS (60-90 seconds, out loud)

I'd use streaming tools and avoid opening the file in an editor. For the top IPs, `awk '{print $1}' access.log | sort | uniq -c | sort -rn | head` reads the file once. To speed it up, I run sort with `LC_ALL=C`, so it uses simple byte comparison, and for compressed or many files I use `zcat` or `zgrep`. For errors, I filter first, for example on status 500, so later steps process less data. On a production server I'd use `nice` and `ionice` to avoid hurting the service, or copy the file to another machine. For very large or repeated analysis, I'd use `GNU parallel`, or better, send logs to a log platform such as Loki or Elasticsearch, so I query indexed data instead of scanning raw files.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `nice -n 19 ionice -c3 awk '{print $1}' access.log \| sort \| uniq -c \| sort -rn \| head` | Top client IPs while being gentle to the production server. |
| 2 | `LC_ALL=C sort` | Prefix the sort commands: faster byte-order sorting instead of locale rules. |
| 3 | `awk '$9 >= 500 {print $9, $7}' access.log \| sort \| uniq -c \| sort -rn \| head` | Most common server error status and URL. |
| 4 | `grep -c ' 500 ' access.log` | Quick count of one specific status code. |
| 5 | `zgrep -h ' 500 ' access.log*.gz \| wc -l` | Search compressed rotated logs without extracting them. |
| 6 | `awk '{print $4}' access.log \| cut -c2-15 \| uniq -c` | Requests per hour (timestamp trimmed) to find spikes. |
| 7 | `split -l 5000000 access.log part_` | Cut into chunks, then process them in parallel on spare cores or another machine. |
| 8 | Ship logs to Loki, Elasticsearch or CloudWatch | Indexed queries beat scanning raw files every time. |

## KEY POINTS TO MENTION

Streaming tools read the file once and use little memory. Filter early, sort late. Locale and compression affect speed. Be careful on production: use low priority or copy the file elsewhere. For ongoing analysis, centralised logging is the right design.

## FIX AND PREVENTION

Keep log formats consistent (JSON helps), rotate and compress logs, ship them to a central system, and build dashboards for the common questions, so nobody needs to grep 10 GB files.

## FOLLOW-UP QUESTIONS THEY MAY ASK

**Q. Why does sort need to come before uniq -c?**
A. uniq only collapses adjacent identical lines, so the input must be sorted to bring duplicates together.

**Q. Why does LC_ALL=C speed up sort?**
A. It avoids slow locale-aware collation and compares bytes directly.

## AVOID THIS MISTAKE

Opening a huge log in vim or less on a busy production server.

# 10 DELETE FILES OLDER THAN 30 DAYS

ADMIN & SECURITY DIFFICULTY ✦ ✦ ✦

## THE QUESTION

**Disk usage is growing because of old temp files. How do you safely delete files older than 30 days in a directory?**

## SAY IT LIKE THIS (60-90 seconds, out loud)

I do it in two stages: preview, then delete. First I run `find` with the same criteria but only printing, to see exactly what would be removed, for example `find /data/tmp -type f -mtime +30 -print`. I check the count and sizes, spot-check the names, and make sure I'm in the right path and that nothing that must be kept is included. Then I run the delete with `-delete`, or `-print0 | xargs -0 rm`, which handles spaces and special characters in names. I add `-xdev` so it doesn't cross into other filesystems, and `-type f` so directories aren't touched. For safety I log what was deleted, run it as a regular user where possible, and never use a variable path that could be empty. Finally I automate it with a cron job, a systemd timer, `tmpfiles.d` for temp directories, or application-level retention.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `find /data/tmp -xdev -type f -mtime +30 -print \| head` | Preview: files older than 30 days, on this filesystem only. |
| 2 | `find /data/tmp -xdev -type f -mtime +30 -printf '%s\n' \| awk '{s+=$1} END {print s/1048576}'` | How many megabytes would be freed? |
| 3 | `find /data/tmp -xdev -type f -mtime +30 -print > /var/log/cleanup.log` | Keep a record of what is going to be removed. |
| 4 | `find /data/tmp -xdev -type f -mtime +30 -delete` | Delete, after verifying the preview. (-delete acts immediately.) |
| 5 | `find /data/tmp -xdev -type f -mtime +30 -print0 \| xargs -0 rm -f` | Alternative that is safe with spaces and newlines in names. |
| 6 | `find /data/tmp -xdev -type d -empty -delete` | Then remove directories that are now empty. |
| 7 | `-mtime +30 versus -mmin +43200 versus -newermt '30 days ago'` | Time options: days, minutes, or a date comparison. |
| 8 | Automate: tmpfiles.d, logrotate, cron or a systemd timer | So nobody has to remember to clean up by hand. |

## KEY POINTS TO MENTION

Always preview before deleting. Use `-type f`, `-xdev` and a fixed path. `-mtime +30` means older than 30 full days. Handle special characters with -print0 and xargs -0. Make sure variables can't expand to an empty or root path.

## FIX AND PREVENTION

Automate retention with `systemd-tmpfiles`, logrotate or application settings, alert on disk growth, and keep backups for anything valuable. Test the command on a copy first.

## FOLLOW-UP QUESTIONS THEY MAY ASK

**Q. What does -mtime +30 mean exactly?**

A. Modified more than 30 full 24-hour periods ago. A file that is 30 days and 20 hours old does not match. It needs to be at least 31 days old.

**Q. How can rm -rf $DIR/ go wrong?**

A. If the variable is empty it can become rm -rf /. Use `set -u`, quotes and checks before deleting.

## AVOID THIS MISTAKE

Running the delete command first and never previewing.

# 11 COPY 2 TB BETWEEN SERVERS SAFELY

DEVOPS & CLOUD DIFFICULTY ✦✦✦

THE QUESTION

**You need to copy 2 TB of data to another server over the network, safely and with the ability to resume if it is interrupted. How do you do it?**

SAY IT LIKE THIS (60-90 seconds, out loud)

I'd use `rsync` over SSH, because it only copies what's different, can resume with `--partial`, and can verify files. A typical command is `rsync -aHAXv --partial --progress -e ssh /data/ user@host:/data/`. The `-a` preserves permissions, times and links, and `-H`, `-A`, `-X` keep hard links, ACLs and extended attributes. I run it inside `tmux`, so it survives a dropped session, and use `--bwlimit` to avoid saturating the network. For a live system I do a first full pass while the service runs, then a short final pass with the service stopped, to catch the changes. After that I verify with a checksum comparison, `rsync -c --dry-run`. For huge datasets or low bandwidth, alternatives are `tar` over SSH, parallel rsync, `rclone` for object storage, or cloud snapshots and data transfer services.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `tmux new -s copy` | Run inside tmux so a dropped SSH session doesn't stop the transfer. |
| 2 | `rsync -aHAXv --partial --progress -e ssh /data/ user@host:/data/` | Archive mode with hard links, ACLs and xattrs. Resumable. |
| 3 | `rsync -aHAX --bwlimit=50000 -e ssh /data/ user@host:/data/` | Limit bandwidth (KB/s) to protect other traffic. |
| 4 | `rsync -aHAX --delete -e ssh /data/ user@host:/data/` | Final pass to catch changes. --delete mirrors removals, so use with care. |
| 5 | `rsync -aHAXc --dry-run -e ssh /data/ user@host:/data/` | Verify with checksums without changing anything. |
| 6 | `tar cf - /data \| pv \| ssh user@host 'tar xf - -C /'` | Stream through tar with a progress bar. Good for many small files. |
| 7 | `ssh -c aes128-gcm@openssh.com user@host` | Faster cipher choice on CPU-limited links (modern CPUs have AES hardware). |
| 8 | **Alternatives: rclone for object storage, cloud snapshots, AWS DataSync, a physical disk** | For huge data, low bandwidth or cloud-to-cloud transfers. |

KEY POINTS TO MENTION

rsync transfers only differences and is restartable. Trailing slashes matter: `src/` copies the contents, `src` copies the directory itself. Use two passes for live data. Verify afterwards, and protect production bandwidth.

FIX AND PREVENTION

Plan the final cut-over: freeze writes, run the last sync, verify, then switch. Document the commands, test on a small subset first, and keep the source intact until the copy is verified.

FOLLOW-UP QUESTIONS THEY MAY ASK

**Q. What does the trailing slash do in rsync?**

A. `src/` copies the contents of src into dest. `src` copies the directory itself, giving `dest/src`.

**Q. Why use --partial?**

A. It keeps partially transferred files, so an interrupted copy resumes instead of starting that file again.

AVOID THIS MISTAKE

Deleting the source before verifying the copy.

# 12 ADD A NEW DISK AND MOUNT IT

ADMIN & SECURITY DIFFICULTY ✦✦✦

**THE QUESTION**

**A new disk has been attached to a Linux server. How do you partition it, format it, and mount it permanently?**

**SAY IT LIKE THIS (60-90 seconds, out loud)**

First I identify the new disk with `lsblk`, being careful not to touch the wrong device. Then I create a partition table and partition: GPT with `parted` or `fdisk` for most disks. I format with a filesystem that suits the workload, ext4 or XFS for general use, using `mkfs`. Then I create a mount point, mount it, and verify with `df -h`. To make it permanent, I find the filesystem's UUID with `blkid` and add a line to `/etc/fstab` using the UUID, not the device name, since names can change. I include sensible options such as `defaults,noatime`, and `nofail` if the system should still boot when the disk is missing. Then I test with `mount -a` before rebooting, set ownership and permissions, and consider LVM if I'll need to grow it later.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `lsblk -f and sudo fdisk -l` | Identify the new, empty disk (for example /dev/sdb) and double-check its size and name. |
| 2 | `sudo parted /dev/sdb --script mklabel gpt mkpart primary ext4 0% 100%` | Create a GPT partition table and one partition covering the disk. |
| 3 | `sudo mkfs.ext4 /dev/sdb1 (or mkfs.xfs /dev/sdb1)` | Create the filesystem. This destroys any data on the partition. |
| 4 | `sudo mkdir -p /data && sudo mount /dev/sdb1 /data` | Create the mount point and mount it for a first test. |
| 5 | `sudo blkid /dev/sdb1` | Get the UUID for fstab. |
| 6 | `echo 'UUID=xxxx-xxxx /data ext4 defaults,noatime,nofail 0 2' \| sudo tee -a /etc/fstab` | Persistent mount using the UUID. `nofail` lets boot continue if the disk is absent. |
| 7 | `sudo mount -a and findmnt --verify` | Test the fstab entry before rebooting. |
| 8 | `sudo chown appuser:appgroup /data and df -hT /data` | Set ownership, and confirm size and filesystem type. |

**KEY POINTS TO MENTION**

Use UUIDs in fstab. Test with `mount -a`. mkfs is destructive, so verify the device name. Consider LVM for flexibility. Add `nofail` for non-critical data disks.

**FIX AND PREVENTION**

Automate disk setup (cloud-init, Ansible), monitor capacity, and use LVM or cloud volume resizing for growth. Plan snapshots and backups for the new volume from day one.

**FOLLOW-UP QUESTIONS THEY MAY ASK**

**Q. What does the last number in an fstab line mean?**

A. The fsck pass order: 1 for the root filesystem, 2 for others checked after it, and 0 means do not check at boot.

**Q. Why GPT instead of MBR?**

A. GPT supports disks over 2 TB and many more partitions, and stores redundant headers for safety.

**AVOID THIS MISTAKE**

Running mkfs on the wrong device.

# 13 FILESYSTEM BECAME READ-ONLY

TROUBLESHOOTING DIFFICULTY ✦✦✦

**THE QUESTION**

**Applications suddenly fail with 'Read-only file system', although nobody changed any mount options. What happened and what do you do?**

**SAY IT LIKE THIS (60-90 seconds, out loud)**

Linux can remount a filesystem read-only when it detects serious errors, to protect data, if the mount option is `errors=remount-ro`, the default on many ext4 systems. So it's a symptom of an underlying problem. I check `dmesg` and the journal for I/O errors or filesystem error messages, which point to the disk, the controller, a cloud volume problem, or filesystem corruption. I confirm with `mount` that it's `ro`, and check SMART or the cloud volume health. Before attempting a repair I make sure I have a backup or snapshot of the data. Recovery is to stop the applications, unmount if possible, and run `fsck` on the unmounted filesystem, or boot to rescue mode for the root filesystem, then remount read-write. If errors recur, I replace the disk or volume. I also check the storage layer, because a hypervisor, SAN or network storage might have interrupted.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `findmnt -o TARGET,OPTIONS \| grep -w ro` | Which filesystems are mounted read-only? |
| 2 | `dmesg -T \| grep -iE 'error\|remount\|i/o\|ext4\|xfs'` | The trigger: I/O errors, journal aborts, or 'Remounting filesystem read-only'. |
| 3 | `journalctl -k -b \| tail -50` | Kernel messages with timestamps around the incident. |
| 4 | `sudo smartctl -a /dev/sda` | Is the underlying disk failing? (In the cloud, check the volume status instead.) |
| 5 | Take a backup or snapshot first | Repairs can lose data, so protect what you have. |
| 6 | `sudo umount /data && sudo fsck -f /dev/sdb1` | Check and repair an unmounted filesystem. (XFS: xfs_repair.) |
| 7 | `sudo mount -o remount,rw /data` | Remount read-write once healthy. For the root filesystem, use rescue mode. |
| 8 | `tune2fs -l /dev/sdb1 \| grep -i 'errors behavior'` | How the filesystem reacts to errors: continue, remount-ro or panic. |

**COMMON ROOT CAUSES**

A failing disk or bad sectors, a loose cable or controller problem, cloud volume or SAN interruptions, filesystem corruption after a crash or power loss, and journal errors.

**FIX AND PREVENTION**

Restore the storage layer, repair the filesystem, and verify the data. Use redundancy (RAID, replicated storage), regular backups, SMART and filesystem error monitoring, and alert on 'Remounting filesystem read-only' messages.

**FOLLOW-UP QUESTIONS THEY MAY ASK**

**Q. What does errors=remount-ro do?**

A. If the kernel detects a filesystem error, it remounts read-only to avoid more damage, instead of continuing or panicking.

**Q. Can you run fsck on a mounted filesystem?**

A. Not safely. Check an unmounted (or read-only) filesystem, otherwise you risk corruption.

**AVOID THIS MISTAKE**

Forcing the filesystem back to read-write without understanding why it went read-only.

# 14 CONNECTIONS DROP DURING BURSTS

TROUBLESHOOTING DIFFICULTY ✦✦✦

## THE QUESTION

**A web server drops connections or clients see timeouts during traffic bursts, although CPU is low. How do you check whether the listen queue is overflowing?**

## SAY IT LIKE THIS (60-90 seconds, out loud)

When a server can't accept connections fast enough, the kernel queues them, first in the SYN queue during the handshake, then in the accept queue until the application calls `accept()`. If the accept queue is full, the kernel drops or ignores new connections, and clients see timeouts or retries. I check with `ss -ltn`: for listening sockets, Recv-Q shows the current queue length and Send-Q the maximum backlog. Then `nstat` or `netstat -s` shows counters such as ListenOverflows and ListenDrops. The queue size is the lower of the application's backlog and `net.core.somaxconn`, and SYN handling depends on `tcp_max_syn_backlog` and SYN cookies. The fixes are raising the application's backlog, for example in nginx's `listen ... backlog=`, raising `somaxconn`, and fixing the real issue: workers that are too slow or too few to accept quickly.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `ss -ltn` | For LISTEN sockets: Recv-Q is connections waiting in the accept queue, Send-Q is the maximum backlog. |
| 2 | `nstat -az \| grep -iE 'ListenOverflows\|ListenDrops'` | Counters that increase when the accept queue overflows. |
| 3 | `netstat -s \| grep -iE 'overflow\|SYNs to LISTEN'` | The older equivalent of the same counters. |
| 4 | `sysctl net.core.somaxconn net.ipv4.tcp_max_syn_backlog net.ipv4.tcp_syncookies` | Kernel limits for the accept queue, SYN queue and SYN cookies. |
| 5 | `sudo sysctl -w net.core.somaxconn=4096` | Raise the kernel cap on the accept queue (the app backlog must be raised too). |
| 6 | `listen 80 backlog=4096; (nginx)` | Application-level backlog. The effective value is the lower of the two. |
| 7 | `top and ss -s` | Are workers busy or blocked? Too few workers means slow accept(). |
| 8 | `dmesg \| grep -i 'SYN flooding'` | The kernel warns if it enabled SYN cookies because the SYN queue was full. |

## COMMON ROOT CAUSES

Too few or busy application workers, a backlog set too low (often 128 or 511), `somaxconn` too small, a very bursty traffic pattern, a SYN flood, slow downstream dependencies blocking the app.

## FIX AND PREVENTION

Raise the backlog and `somaxconn`, increase workers or use an event-driven server, add a load balancer to absorb bursts, protect against SYN floods (SYN cookies, rate limits), and alert on ListenOverflows.

## FOLLOW-UP QUESTIONS THEY MAY ASK

**Q. SYN queue versus accept queue?**

A. The SYN queue holds half-open connections during the handshake. The accept queue holds established connections waiting for the application to accept them.

**Q. What are SYN cookies?**

A. A technique that avoids storing state for half-open connections when the SYN queue is full, so a SYN flood doesn't exhaust memory.

## AVOID THIS MISTAKE

Raising somaxconn alone while the application's backlog stays small.

# 15 WRITE A PRODUCTION-READY SYSTEMD UNIT

DEVOPS & CLOUD DIFFICULTY ✦✦✦

**THE QUESTION**

**You need to run a custom application as a service. How would you write a production-ready systemd unit?**

**SAY IT LIKE THIS (60-90 seconds, out loud)**

I'd write a unit with a dedicated unprivileged user, a clear start command, a restart policy, dependencies and hardening. The `[Unit]` section sets a description and ordering, such as `After=network-online.target`. In `[Service]` I set `User` and `Group`, `ExecStart` with an absolute path, an `EnvironmentFile` for configuration, `Restart=on-failure` with `RestartSec`, and limits like `LimitNOFILE` and `MemoryMax`. For safety I add sandboxing: `NoNewPrivileges=true`, `ProtectSystem=strict`, `ProtectHome=true`, `PrivateTmp=true`, and `ReadWritePaths` only where needed. Logging goes to the journal by default. In `[Install]` I use `WantedBy=multi-user.target`. Then I run `systemd-analyze verify`, `daemon-reload` and `enable --now`, check the status and logs, and review the hardening score with `systemd-analyze security`.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `sudo useradd --system --no-create-home --shell /usr/sbin/nologin appuser` | A dedicated service user that cannot log in. |
| 2 | [Unit] After=network-online.target Wants=network-online.target | Wait for the network, and declare dependencies. |
| 3 | [Service] User=appuser ExecStart=/opt/app/bin/app EnvironmentFile=/etc/app/app.env Restart=on-failure RestartSec=5 LimitNOFILE=65536 | Run as a dedicated user, restart on failure, load config, raise file limits. |
| 4 | Sandbox: NoNewPrivileges=true ProtectSystem=strict ProtectHome=true PrivateTmp=true ReadWritePaths=/var/lib/app | Least privilege for the process. |
| 5 | [Install] WantedBy=multi-user.target | Start at boot in normal multi-user mode. |
| 6 | `systemd-analyze verify /etc/systemd/system/app.service` | Catch syntax errors and missing paths. |
| 7 | `sudo systemctl daemon-reload && sudo systemctl enable --now app` | Load the unit, enable it at boot and start it now. |
| 8 | `systemctl status app and journalctl -u app -f and systemd-analyze security app.service` | State, live logs, and the hardening score. |

**KEY POINTS TO MENTION**

Type= must match how the app behaves (simple, forking, notify). Order with After= and use Wants= or Requires= for dependencies. Use Restart= with RestartSec. Sandbox with NoNewPrivileges, ProtectSystem and PrivateTmp. Never run services as root unless necessary.

**FIX AND PREVENTION**

Keep unit files in version control or configuration management, use drop-ins (`systemctl edit`) for overrides, monitor restart counts, and test restart and reboot behaviour.

**FOLLOW-UP QUESTIONS THEY MAY ASK**

**Q. Type=simple versus forking versus notify?**

A. simple: the process started by ExecStart is the service. forking: it forks and the parent exits. notify: the service tells systemd when it is ready (sd_notify).

**Q. Why use drop-ins?**

A. `systemctl edit` writes override files, so you change only what you need and package upgrades don't overwrite your changes.

**AVOID THIS MISTAKE**

Running the service as root with no limits and no sandboxing.

## 16 MIGRATE A SERVER TO THE CLOUD

DEVOPS & CLOUD DIFFICULTY ✦✦✦

**THE QUESTION**

**You must migrate an on-premises Linux server running a business application to the cloud with minimal downtime. How do you plan and execute it?**

**SAY IT LIKE THIS (60-90 seconds, out loud)**

I'd treat it as a planned project in phases. First, discovery: what runs on the server, its dependencies, ports, data size, traffic patterns, and the acceptable downtime. Then I pick the strategy: rehost (lift and shift) to a VM for speed, replatform (managed database or load balancer) for more benefit, or refactor later. I'd build the target environment as code, with networking, security groups and monitoring. For data, I do an initial bulk copy while the source still runs, then repeated incremental syncs, and for databases I use replication. I rehearse with a test migration and a rollback plan, lower DNS TTLs ahead of time, then do the cut-over in a window: stop writes, run the last sync, verify, switch DNS or the load balancer, and watch the metrics. I keep the old server for a rollback period.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | **Discovery: inventory services, ports, dependencies, data size, traffic, compliance** | Know what you are moving before you move it. |
| 2 | `ss -tulpn and lsof -i -P -n and systemctl list-unit-files --state=enabled` | What listens, what talks to what, and what starts at boot. |
| 3 | **Choose the strategy: rehost, replatform or refactor** | Match speed, risk and benefit. |
| 4 | `terraform apply` | Build the cloud environment (network, instances, security groups, backups, monitoring) as code, so it is repeatable. |
| 5 | `rsync -aHAX --partial -e ssh /data/ user@new:/data/` | Initial bulk copy, then repeated syncs to shrink the final delta. |
| 6 | **Databases: replication or a migration service (for example streaming replication)** | Keep the new database in sync, then promote it at cut-over. |
| 7 | **Lower the DNS TTL days before. Rehearse. Prepare the rollback** | Fast switch-over and a safe way back. |
| 8 | **Cut-over: stop writes, final sync, verify, switch DNS or LB, monitor** | Then keep the old server for a defined rollback window. |

**KEY POINTS TO MENTION**

Discovery and dependencies prevent surprises. Rehearse the migration and write the rollback plan. Minimise downtime with incremental sync and replication. Lower DNS TTLs in advance. Validate with checks and monitoring after cut-over.

**FIX AND PREVENTION**

Document everything, automate the target environment, define success criteria (latency, errors, data checks), and decommission the old server only after a stable period and a final backup.

**FOLLOW-UP QUESTIONS THEY MAY ASK**

**Q. Rehost versus replatform versus refactor?**

A. Rehost moves the app as is. Replatform makes small cloud optimisations (managed database, load balancer). Refactor redesigns the app for cloud-native services.

**Q. Why lower the DNS TTL before cut-over?**

A. So clients pick up the new address quickly, and a rollback also propagates fast.

**AVOID THIS MISTAKE**

Migrating without a rehearsal and a rollback plan.

# 17 THOUSANDS OF FAILED SSH LOGINS

ADMIN & SECURITY DIFFICULTY ✦✦✦

## THE QUESTION

**Your auth log shows thousands of failed SSH login attempts from many IP addresses. What do you do?**

## SAY IT LIKE THIS (60-90 seconds, out loud)

First I assess: is it background noise from internet scanners, which is normal for any exposed SSH, or is someone succeeding or targeting a specific user? I check for successful logins from the same IPs, with `last` and by searching for 'Accepted' in the log. If anything succeeded, I treat it as an incident. Then I reduce the exposure: allow SSH only from known IP ranges through the firewall or security group, put it behind a VPN or bastion, or use SSM or similar so port 22 isn't public at all. On the server I disable password authentication and root login, so brute force can't succeed, and I install fail2ban or similar to ban repeat offenders. Changing the port only reduces noise, it's not real security. I'd add MFA for admins and alerts for successful logins from unusual sources.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `sudo grep 'Failed password' /var/log/auth.log \| awk '{print $(NF-3)}' \| sort \| uniq -c \| sort -rn \| head` | Top attacking IPs. (RHEL: /var/log/secure.) |
| 2 | `sudo grep 'Accepted' /var/log/auth.log \| tail` and `last -a \| head` | Did any login succeed? A success turns this into an incident. |
| 3 | `sudo sshd -T \| grep -iE 'passwordauthentication\|permitrootlogin\|maxauthtries'` | Check that password login and root login are off. |
| 4 | `sudo ufw allow from 203.0.113.0/24 to any port 22 proto tcp` | Restrict SSH to trusted networks (or a security group rule in the cloud). |
| 5 | `sudo apt install fail2ban` and `sudo fail2ban-client status sshd` | Ban IPs after repeated failures, and show current bans. |
| 6 | `PasswordAuthentication no` and `PermitRootLogin no (sshd_config)` | Make brute force useless. Keys only. |
| 7 | Bastion host, VPN, SSM Session Manager, or MFA | Remove direct public exposure of SSH where possible. |
| 8 | Central logging with alerts on successful logins from new IPs or countries | Detect real compromise, not just noise. |

## KEY POINTS TO MENTION

Failed attempts are normal internet noise. Successful logins are what matter. Keys-only authentication makes brute force ineffective. Reduce exposure, add rate limiting, and alert on anomalies. Changing the port only reduces log noise.

## FIX AND PREVENTION

Use key-based login, restrict by source, deploy fail2ban or an equivalent, use a bastion or SSM, add MFA, and monitor. Keep the process documented and tested.

## FOLLOW-UP QUESTIONS THEY MAY ASK

**Q. Does changing the SSH port improve security?**

A. Only slightly: it reduces log noise from scanners, but it's security through obscurity. Keys-only login and source restrictions are the real controls.

**Q. What does fail2ban do?**

A. It watches logs for repeated failures and adds temporary firewall rules to ban offending IP addresses.

## AVOID THIS MISTAKE

Assuming the noise is harmless without checking for successful logins.

# 18 AUDIT FOR PRIVILEGE ESCALATION

ADMIN & SECURITY DIFFICULTY ✦✦✦

THE QUESTION

**You are asked to audit a Linux server for privilege-escalation risks. What do you look for, and how?**

SAY IT LIKE THIS (60-90 seconds, out loud)

I look at the ways an ordinary user could become root. First, sudo: `sudo -l` for each user, looking for broad rights like `ALL`, commands that can spawn shells or edit files, and NOPASSWD rules. Second, SUID and SGID binaries: `find / -perm -4000`, compared against a baseline, checking for unusual or writable ones. Third, Linux capabilities with `getcap -r /`. Fourth, writable files and directories that root executes or loads: cron jobs and scripts, systemd units, PATH directories, and world-writable files. I also review kernel and package versions for known vulnerabilities, services running as root that don't need to, credentials in files or history, and group memberships like docker or lxd, which are effectively root. For the findings I give a risk rating and fixes, and I use a baseline tool such as Lynis.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `sudo -l and sudo grep -r NOPASSWD /etc/sudoers /etc/sudoers.d/` | What can users run as root? Look for ALL, shells, editors and NOPASSWD. |
| 2 | `find / -perm -4000 -type f 2>/dev/null` | SUID binaries. Compare with a known-good list and check unusual ones. |
| 3 | `find / -perm -2000 -type f 2>/dev/null and getcap -r / 2>/dev/null` | SGID files, and files with Linux capabilities. |
| 4 | `find / -xdev -type f -perm -0002 2>/dev/null` | World-writable files. |
| 5 | `ls -l /etc/cron* /etc/systemd/system and crontab -l` | Scripts run by root that a normal user could modify. |
| 6 | `echo $PATH \| tr ':' '\n' \| xargs ls -ld` | Is any directory in root's PATH writable by others? |
| 7 | `getent group docker lxd sudo wheel` | Who is in groups that grant root-equivalent access? |
| 8 | `uname -r and apt list --upgradable and sudo lynis audit system` | Kernel and package versions, and a general hardening baseline. |

COMMON ROOT CAUSES

Over-broad sudo rules, unneeded SUID binaries, writable scripts run by root, docker group membership, weak file permissions, outdated kernels or packages, secrets stored in plain text, misconfigured capabilities.

FIX AND PREVENTION

Apply least privilege, remove unneeded SUID bits, fix permissions, restrict sudo to specific commands, keep systems patched, and use a baseline tool plus regular reviews. Automate the checks.

FOLLOW-UP QUESTIONS THEY MAY ASK

**Q. Why is docker group membership dangerous?**

A. Anyone who can run docker can start a container that mounts the host filesystem, which gives root access to the host.

**Q. What is GTFOBins?**

A. A curated list of Unix binaries that can be abused for privilege escalation when allowed through sudo or SUID. It is useful when auditing.

AVOID THIS MISTAKE

Only running a scanner and not reviewing the results by hand.

# 19 FORK: RETRY: RESOURCE UNAVAILABLE

TROUBLESHOOTING DIFFICULTY ✦✦✦

## THE QUESTION

**Users get 'fork: retry: Resource temporarily unavailable' and cannot start new commands, and a service cannot create threads. What is happening?**

## SAY IT LIKE THIS (60-90 seconds, out loud)

That error means the system or the user has hit a limit on processes or threads, so `fork()` fails with EAGAIN. It could be a runaway process creating children, a fork bomb, a thread leak in an application, or limits set too low. I check which limit applies: the per-user `ulimit -u` (nproc), the system-wide `kernel.pid_max` and `kernel.threads-max`, the cgroup `pids.max` for the service or container, and memory exhaustion that prevents creating new tasks. I count processes and threads per user and per process with `ps` to find who is creating them. If I can't log in, I use an already-open root shell or the console. To fix it, I stop the runaway source, then set sensible limits: `nproc` in `limits.d`, `TasksMax` in systemd units, and `--pids-limit` for containers, and I fix the code that leaks threads.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `ulimit -u and cat /etc/security/limits.d/*` | The per-user process limit for this shell, and the configured limits. |
| 2 | `ps -eo user= \| sort \| uniq -c \| sort -rn \| head` | Processes per user. One user far above the others is the culprit. |
| 3 | `ps -eLf \| wc -l` | Total number of threads on the system. |
| 4 | `cat /proc/sys/kernel/pid_max and cat /proc/sys/kernel/threads-max` | System-wide limits on PIDs and threads. |
| 5 | `systemctl show app -p TasksMax` | The systemd limit on tasks per service (the cgroup pids.max). |
| 6 | `ps -eo pid,ppid,nlwp,cmd --sort=-nlwp \| head` | Processes with the most threads (nlwp is the number of threads). |
| 7 | `sudo pkill -u baduser` | Contain a runaway source, using an existing root shell or the console. |
| 8 | `echo 'baduser hard nproc 4096' \| sudo tee /etc/security/limits.d/90-nproc.conf` | Set a per-user limit. Also use TasksMax= in units and --pids-limit in Docker. |

## COMMON ROOT CAUSES

A fork bomb or runaway script, a thread leak in an application, per-user nproc limits that are too low, systemd TasksMax or a container pids-limit that is too low, kernel PID or thread limits reached, and memory pressure preventing new tasks.

## FIX AND PREVENTION

Stop the source, set per-user and per-service limits, fix leaks, and alert on process and thread counts. Keep a privileged recovery path (the console or an existing root session).

## FOLLOW-UP QUESTIONS THEY MAY ASK

**Q. What is a fork bomb?**

A. A process that endlessly creates copies of itself until the system runs out of process slots or memory, such as the classic shell one-liner :(){ :|:& };:

**Q. What does TasksMax do in systemd?**

A. It limits the number of tasks (processes and threads) a service's cgroup may create.

## AVOID THIS MISTAKE

Opening a new SSH session while the system is out of PIDs. Use an existing shell or the console.

# 20 CHOOSING EXT4, XFS OR BTRFS

ADMIN & SECURITY DIFFICULTY ✦✦✦

## THE QUESTION

**You are setting up storage for a new workload. How do you choose between ext4, XFS and Btrfs?**

## SAY IT LIKE THIS (60-90 seconds, out loud)

I choose based on the workload, the features needed and the team's operational experience. ext4 is the safe default: mature, well understood, with good tools, and it can be grown and shrunk offline. XFS excels with large files, big filesystems and highly parallel IO, which is why it's the default on RHEL, and it can grow online but cannot shrink. Btrfs, or ZFS, offers snapshots, checksums for data integrity, compression and built-in volume management, which is useful for backups and rollbacks, but it needs more care and has more operational complexity. Mount options matter too: `noatime` to avoid unnecessary writes, and periodic `fstrim` for SSDs. I'd also decide whether to use LVM or the cloud's volume resizing, and I'd benchmark the real workload with `fio` instead of trusting rules of thumb.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `lsblk -f and findmnt -t ext4,xfs,btrfs` | See which filesystems are in use and where. |
| 2 | `mkfs.ext4 /dev/sdb1 or mkfs.xfs /dev/sdb1 or mkfs.btrfs /dev/sdb1` | Create the chosen filesystem (destroys existing data). |
| 3 | `mount -o noatime /dev/sdb1 /data` | Skip access-time updates to reduce writes. |
| 4 | `sudo systemctl enable --now fstrim.timer` | Periodic TRIM for SSDs. Often preferred over the `discard` mount option. |
| 5 | `fio --name=test --filename=/data/t --rw=randrw --bs=4k --size=1G --runtime=30 --time_based` | Benchmark with a workload that matches yours. |
| 6 | `btrfs subvolume snapshot /data /data/.snap` | Instant snapshots and rollbacks, if you use Btrfs. |
| 7 | `xfs_growfs /data and resize2fs /dev/sdb1` | Grow XFS (online) and ext4. XFS cannot shrink. |
| 8 | **Consider ZFS or Btrfs for snapshots and checksums. Keep ext4 or XFS for simplicity** | Match features to needs, and to what your team can operate. |

## KEY POINTS TO MENTION

ext4 is the reliable default. XFS suits large files and parallel IO, and only grows. Btrfs and ZFS bring snapshots, checksums and compression. Match the choice to workload, recovery needs and team skills. Test with realistic workloads, and plan backups regardless.

## FIX AND PREVENTION

Standardise on one or two filesystems, document mount options, automate provisioning, and monitor capacity and errors. Remember that no filesystem replaces backups.

## FOLLOW-UP QUESTIONS THEY MAY ASK

**Q. Why can't XFS shrink?**

A. Its design (allocation groups) doesn't support removing space. You create a smaller filesystem and copy the data.

**Q. What does noatime do?**

A. It stops the kernel updating a file's access time on every read, which reduces writes and improves performance.

## AVOID THIS MISTAKE

Choosing a filesystem because of a blog post, without testing your own workload.

## 21 RAPID-FIRE ROUND

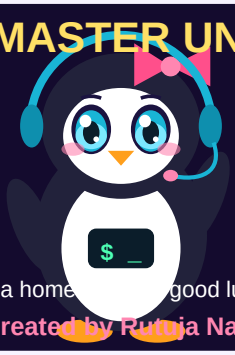
*10 quick questions, 1 to 2 sentence answers*

| | | |
|---|---|---|
| 1 | **What is a system call?** | A controlled entry point where a program asks the kernel to do something privileged, such as open, read or fork. |
| 2 | **User space versus kernel space?** | Kernel space runs with full hardware access. User-space programs run with restricted privileges and ask the kernel through system calls. |
| 3 | **fork versus exec?** | fork creates a copy of a process. exec replaces a process's program with a new one, keeping the same PID. |
| 4 | **Orphan versus zombie process?** | An orphan is a running child whose parent died, and init adopts it. A zombie has finished, but its parent has not collected its exit status. |
| 5 | **What is a kernel module?** | Code loaded into the running kernel on demand, such as a driver. Managed with lsmod, modprobe and rmmod. |
| 6 | **What does chroot do?** | It changes a process's apparent root directory to a subtree, giving basic filesystem isolation. It is not a full security boundary. |
| 7 | **What is tmpfs?** | A filesystem stored in RAM (and swap). Fast and cleared on reboot. Used for /run, /dev/shm and often /tmp. |
| 8 | **TTY versus PTY?** | A TTY is a terminal device. A PTY is a pseudo-terminal pair that emulates one, used by SSH, terminal emulators and containers. |
| 9 | **Buffers versus cache in free?** | Cache holds file contents read from disk. Buffers hold block-device metadata and raw blocks. Both are reclaimable. |
| 10 | **What is copy-on-write?** | Data is shared until someone modifies it, and then a private copy is made. Used by fork, Btrfs, ZFS and container image layers. |

**BEFORE THE INTERVIEW: FINAL CHECKLIST**

- Practise each answer out loud in 60 to 90 seconds. Record yourself once.
- Prepare a real example for each topic: "Last time I saw this, I..."
- Know why a command works, not just its syntax.
- Admit what you don't know, then explain how you would find out.
- End answers with prevention: monitoring, automation, documentation.
- Keep a home lab (VM, Docker, minikube). Break things on purpose and fix them.

## KERNEL MASTER UNLOCKED!




Practice out loud, build a home good luck with your interviews.

**Created by Rutuja Naik**